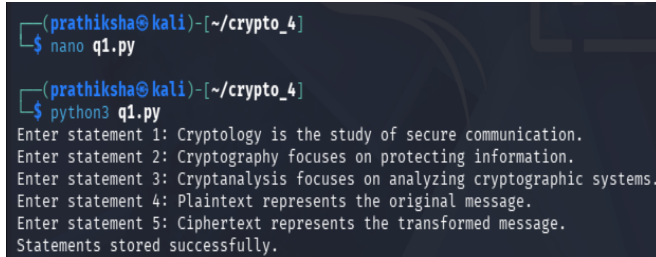


## Cryptology Assignment - 4

1. Write a Python program to accept five statements related to cryptology from the user and store them in a text file named cryptology.txt. Store each statement on a separate line.

```
file = open("cryptology.txt", "w")
for i in range(5):
    statement = input("Enter statement {}: ".format(i + 1))
    file.write(statement + "\n")
file.close()
print("Statements stored successfully.")
```

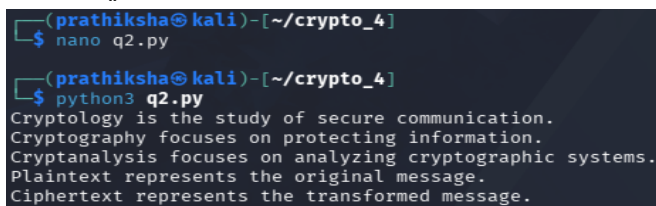


```
(prathiksha@kali)~/crypto_4
$ nano q1.py

(prathiksha@kali)~/crypto_4
$ python3 q1.py
Enter statement 1: Cryptology is the study of secure communication.
Enter statement 2: Cryptography focuses on protecting information.
Enter statement 3: Cryptanalysis focuses on analyzing cryptographic systems.
Enter statement 4: Plaintext represents the original message.
Enter statement 5: Ciphertext represents the transformed message.
Statements stored successfully.
```

2. Write a Python program to read cryptology.txt and display its complete contents.

```
file = open("cryptology.txt", "r")
content = file.read()
print(content)
file.close()
```

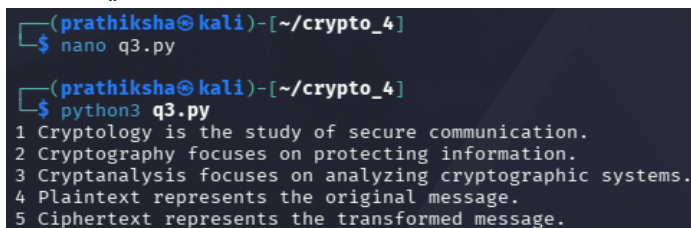


```
(prathiksha@kali)~/crypto_4
$ nano q2.py

(prathiksha@kali)~/crypto_4
$ python3 q2.py
Cryptology is the study of secure communication.
Cryptography focuses on protecting information.
Cryptanalysis focuses on analyzing cryptographic systems.
Plaintext represents the original message.
Ciphertext represents the transformed message.
```

3. Write a program to read cryptology.txt one line at a time and display each line along with its line number.

```
file = open("cryptology.txt", "r")
line_number = 1
for line in file:
    print(line_number, line.strip())
    line_number += 1
file.close()
```



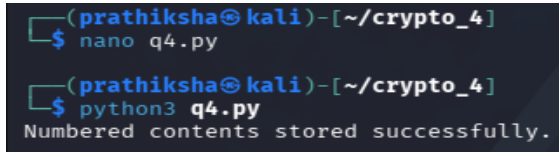
```
(prathiksha@kali)~/crypto_4
$ nano q3.py

(prathiksha@kali)~/crypto_4
$ python3 q3.py
1 Cryptology is the study of secure communication.
2 Cryptography focuses on protecting information.
3 Cryptanalysis focuses on analyzing cryptographic systems.
4 Plaintext represents the original message.
5 Ciphertext represents the transformed message.
```

4. write a program to store the output in a file called cryptology\_no.txt

```
file = open("cryptology.txt", "r")
output = open("cryptology_no.txt", "w")
line_number = 1
for line in file:
    output.write(str(line_number) + " " + line)
    line_number += 1
file.close()
output.close()
```

```
print("Numbered contents stored successfully.")
```



```
(prathiksha@kali)~[/crypto_4]
$ nano q4.py

(prathiksha@kali)~[/crypto_4]
$ python3 q4.py
Numbered contents stored successfully.
```

5. Write a program to read cryptology.txt and determine:

Number of lines

Number of words

Number of characters

```
file = open("cryptology.txt", "r")
```

```
lines = file.readlines()
```

```
number_of_lines = len(lines)
```

```
number_of_words = 0
```

```
number_of_characters = 0
```

```
for line in lines:
```

```
    number_of_words += len(line.split())
```

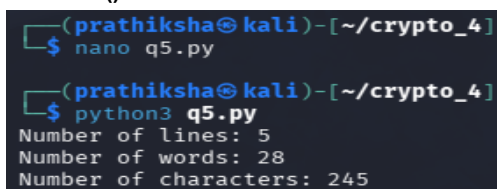
```
    number_of_characters += len(line)
```

```
print("Number of lines:", number_of_lines)
```

```
print("Number of words:", number_of_words)
```

```
print("Number of characters:", number_of_characters)
```

```
file.close()
```



```
(prathiksha@kali)~[/crypto_4]
$ nano q5.py

(prathiksha@kali)~[/crypto_4]
$ python3 q5.py
Number of lines: 5
Number of words: 28
Number of characters: 245
```

6. Read cryptology\_no.txt and count the number of:

Uppercase letters

Lowercase letters

Digits

Spaces

Special characters

```
file = open("cryptology_no.txt", "r")
```

```
uppercase = 0
```

```
lowercase = 0
```

```
digits = 0
```

```
spaces = 0
```

```
special = 0
```

```
for line in file:
```

```
    for ch in line:
```

```
        if ch.isupper():
```

```
            uppercase += 1
```

```
        elif ch.islower():
```

```
            lowercase += 1
```

```
        elif ch.isdigit():
```

```
            digits += 1
```

```
        elif ch.isspace():
```

```
            spaces += 1
```

```
        else:
```

```
            special += 1
```

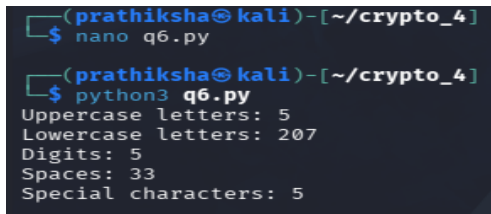
```
print("Uppercase letters:", uppercase)
```

```
print("Lowercase letters:", lowercase)
```

```
print("Digits:", digits)
```

```
print("Spaces:", spaces)
```

```
print("Special characters:", special)
file.close()
```

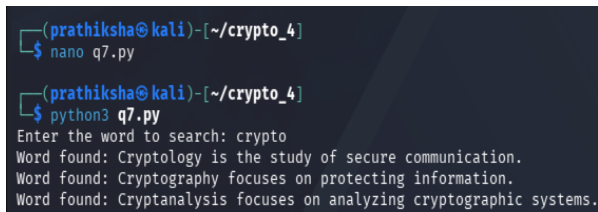


```
(prathiksha@kali)-[~/crypto_4]
$ nano q6.py

(prathiksha@kali)-[~/crypto_4]
$ python3 q6.py
Uppercase letters: 5
Lowercase letters: 207
Digits: 5
Spaces: 33
Special characters: 5
```

7. Write a program that accepts a word from the user and searches for that word in cryptology.txt.

```
word = input("Enter the word to search: ")
file = open("cryptology.txt", "r")
found = False
for line in file:
    if word.lower() in line.lower():
        print("Word found:", line.strip())
        found = True
if not found:
    print("Word not found.")
file.close()
```

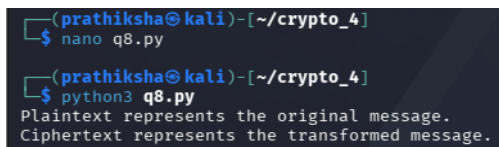


```
(prathiksha@kali)-[~/crypto_4]
$ nano q7.py

(prathiksha@kali)-[~/crypto_4]
$ python3 q7.py
Enter the word to search: crypto
Word found: Cryptology is the study of secure communication.
Word found: Cryptography focuses on protecting information.
Word found: Cryptanalysis focuses on analyzing cryptographic systems.
```

8. Read cryptology.txt and display only the lines containing the word message.

```
file = open("cryptology.txt", "r")
for line in file:
    if "message" in line.lower():
        print(line.strip())
file.close()
```

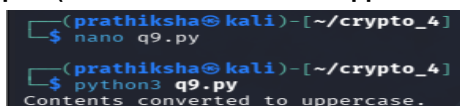


```
(prathiksha@kali)-[~/crypto_4]
$ nano q8.py

(prathiksha@kali)-[~/crypto_4]
$ python3 q8.py
Plaintext represents the original message.
Ciphertext represents the transformed message.
```

9. Write a Python program to read cryptology.txt, convert all alphabetic characters to uppercase, and store the result in a new file called normalized.txt.

```
file = open("cryptology.txt", "r")
output = open("normalized.txt", "w")
for line in file:
    output.write(line.upper())
file.close()
output.close()
print("Contents converted to uppercase.")
```



```
(prathiksha@kali)-[~/crypto_4]
$ nano q9.py

(prathiksha@kali)-[~/crypto_4]
$ python3 q9.py
Contents converted to uppercase.
```

10. Write a program to read the file and print only the lines containing only alphabetic characters from the cryptology\_no.txt file.

```

file = open("cryptology_no.txt", "r")
for line in file:
    line = line.strip()
    parts = line.split(" ", 1)
    if len(parts) == 2:
        text = parts[1]
        if text.replace(" ", "").isalpha():
            print(line)
file.close()

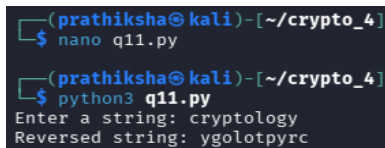
```

11. Write a program to get the string from the user and reverse the string.

```

text = input("Enter a string: ")
reverse = text[::-1]
print("Reversed string:", reverse)

```



```

(prathiksha@kali)~/crypto_4
$ nano q11.py
(prathiksha@kali)~/crypto_4
$ python3 q11.py
Enter a string: cryptology
Reversed string: ygotolpyrc

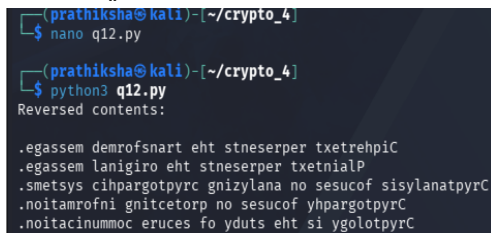
```

12. Improve the above code to reverse the string from the file.

```

file = open("cryptology.txt", "r")
text = file.read()
reverse = text[::-1]
print("Reversed contents:")
print(reverse)
file.close()

```



```

(prathiksha@kali)~/crypto_4
$ nano q12.py
(prathiksha@kali)~/crypto_4
$ python3 q12.py
Reversed contents:
.egassem demrofsnart eht stneserper txetrehpiC
.egassem lanigiro eht stneserper txetnialP
.smetsys cihpargotpyrc gnizylana no sesucof sisylanatpyrc
.noitamrofni gnitcetorp no sesucof yhpargotpyrc
.noitacinummoc eruces fo yduts eht si ygotolpyrc

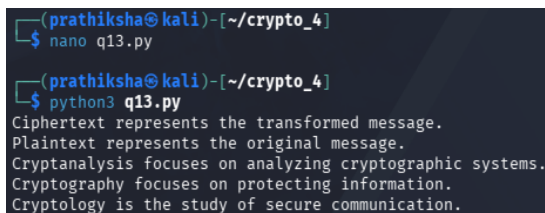
```

13. Simulate the tac command in Python.

```

file = open("cryptology.txt", "r")
lines = file.readlines()
for line in reversed(lines):
    print(line.strip())
file.close()

```



```

(prathiksha@kali)~/crypto_4
$ nano q13.py
(prathiksha@kali)~/crypto_4
$ python3 q13.py
Ciphertext represents the transformed message.
Plaintext represents the original message.
Cryptanalysis focuses on analyzing cryptographic systems.
Cryptography focuses on protecting information.
Cryptology is the study of secure communication.

```

## Level 2: Regex

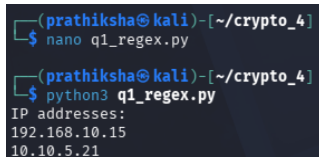
1. Write a Python program to read security\_data.txt and use a regular expression to identify and display all IPv4-address-like patterns present in the file.

```

import re
file = open("security_data.txt", "r")
text = file.read()
ip_addresses = re.findall(r'\b(?:\d{1,3}\.){3}\d{1,3}\b', text)
print("IP addresses:")
for ip in ip_addresses:

```

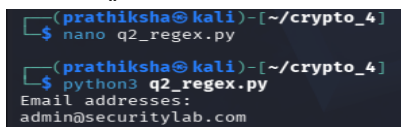
```
print(ip)
file.close()
```



```
(prathiksha@kali) - [~/crypto_4]
$ nano q1_regex.py
(prathiksha@kali) - [~/crypto_4]
$ python3 q1_regex.py
IP addresses:
192.168.10.15
10.10.5.21
```

2. Write a Python program to identify and display all email-address-like patterns present in security\_data.txt.

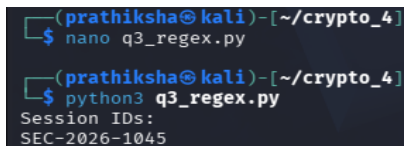
```
import re
file = open("security_data.txt", "r")
text = file.read()
emails = re.findall(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b', text)
print("Email addresses:")
for email in emails:
    print(email)
file.close()
```



```
(prathiksha@kali) - [~/crypto_4]
$ nano q2_regex.py
(prathiksha@kali) - [~/crypto_4]
$ python3 q2_regex.py
Email addresses:
admin@securitylab.com
```

3. Write a Python program to extract session IDs that follow the format SEC-YYYY-NNNN, where YYYY and NNNN represent four digits.

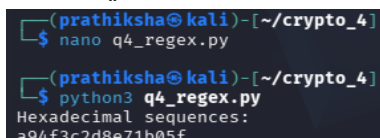
```
import re
file = open("security_data.txt", "r")
text = file.read()
session_ids = re.findall(r'SEC-\d{4}-\d{4}', text)
print("Session IDs:")
for session in session_ids:
    print(session)
file.close()
```



```
(prathiksha@kali) - [~/crypto_4]
$ nano q3_regex.py
(prathiksha@kali) - [~/crypto_4]
$ python3 q3_regex.py
Session IDs:
SEC-2026-1045
```

4. Write a Python program to identify all hexadecimal sequences containing the characters 0–9, a–f, or A–F.

```
import re
file = open("security_data.txt", "r")
text = file.read()
hex_values = re.findall(r'Hash:\s*([0-9a-fA-F]+)', text)
print("Hexadecimal sequences:")
for value in hex_values:
    print(value)
file.close()
```



```
(prathiksha@kali) - [~/crypto_4]
$ nano q4_regex.py
(prathiksha@kali) - [~/crypto_4]
$ python3 q4_regex.py
Hexadecimal sequences:
a94f3c2d8e71b05f
```

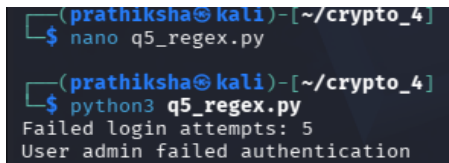
5. Write a Python program to read security\_data.txt and display all lines containing the word failed, irrespective of uppercase or lowercase.

```
import re
file = open("security_data.txt", "r")
for line in file:
```

```

if re.search(r'failed', line, re.IGNORECASE):
    print(line.strip())
file.close()

```



```

(prathiksha@kali) - [~/crypto_4]
$ nano q5_regex.py

(prathiksha@kali) - [~/crypto_4]
$ python3 q5_regex.py
Failed login attempts: 5
User admin failed authentication

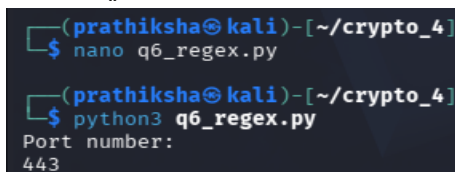
```

6. Write a Python program to identify and extract the port number from statements of the form Port 443 connection established.

```

import re
file = open("security_data.txt", "r")
text = file.read()
port = re.findall(r'Port\s+(\d+)\s+connection established', text)
print("Port number:")
for p in port:
    print(p)
file.close()

```



```

(prathiksha@kali) - [~/crypto_4]
$ nano q6_regex.py

(prathiksha@kali) - [~/crypto_4]
$ python3 q6_regex.py
Port number:
443

```

Write a Python program to read the contents of security\_data.txt and use regular expressions to extract IP-address-like patterns, email addresses, session IDs, and hexadecimal sequences. Store the identified patterns in a new file named pattern\_results.txt.

```

import re
file = open("security_data.txt", "r")
text = file.read()
# Extract IP addresses
ip_addresses = re.findall(r'\b(?:\d{1,3}\.){3}\d{1,3}\b', text)
# Extract email addresses
emails = re.findall(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b', text)
# Extract session IDs
session_ids = re.findall(r'SEC-\d{4}-\d{4}', text)
# Extract hexadecimal sequences
hex_values = re.findall(r'Hash:\s*([0-9a-fA-F]+)', text)
file.close()
# Store results in pattern_results.txt
output = open("pattern_results.txt", "w")
output.write("IP Addresses:\n")
for ip in ip_addresses:
    output.write(ip + "\n")
output.write("\nEmail Addresses:\n")
for email in emails:
    output.write(email + "\n")
output.write("\nSession IDs:\n")
for session in session_ids:
    output.write(session + "\n")
output.write("\nHexadecimal Sequences:\n")
for value in hex_values:
    output.write(value + "\n")
output.close()
print("Patterns extracted successfully.")
print("Results stored in pattern_results.txt")

```

```
(prathiksha@kali)-[~/crypto_4]  
$ nano combined_regex.py  
  
(prathiksha@kali)-[~/crypto_4]  
$ python3 combined_regex.py  
Patterns extracted successfully.  
Results stored in pattern_results.txt
```